



C++ Programming Language

Presented By

Usha Rani Sargurunathan

STG Group

Functions

Functions are blocks of code designed to perform a specific task. They help organize code, improve reusability, and make programs easier to read.

Function Concepts

- **Function Prototype:** A declaration that tells the compiler about a function's name, return type, and parameters before it's used.
- **Inline Function:** A **hint** to the compiler to **substitute the function call with the function's body** to reduce call overhead. This is generally for small functions.
- **Recursive Function:** A function that **calls itself to solve a problem** by breaking it down into smaller, identical subproblems.
- **Overloading Functions:** Creating multiple functions with **the same name but different parameter lists**.
- **Pass by Value vs. Pass by Reference:**
 - **Pass by value** copies the argument's data, so changes inside the function don't affect the original variable.
 - **Pass by reference** passes an alias to the original variable, so changes inside the function **do** affect the original.
- **Default Arguments:** Parameters that have a default value if the caller doesn't provide one.

Function Prototype

- **Definition:** A declaration of a function that specifies its **name, return type, and the types and order of its parameters**. It does *not* include the function's body.
- **Purpose:**
 - To inform the compiler about a function before its actual definition.
 - Allows functions to be called before they are defined in the source code.
 - Enables type checking for function calls, catching errors at compile time.
- **Syntax:** `returnType functionName(parameterType1, parameterType2, ...);`

Recursive Function

- **Definition:** A function that calls itself, either directly or indirectly, to solve a problem.
- **Mechanism:**
 - **Base Case:** A condition that stops the recursion; without it, the function would call itself indefinitely (infinite recursion).
 - **Recursive Step:** The part where the function calls itself with a modified input, moving closer to the base case.
- **Examples:** Calculating factorials, Fibonacci sequence.
- **Caution:** Can lead to **stack overflow** if the base case is not reached or if the recursion depth is too large.

Example - Factorial

```
# include<iostream>
using namespace std;
/*program for recursive function
*/
void fun(int n)
{
    if(n>0)
    {
        cout<<n<<endl;
        fun(n-1);
    } }
int main()
{
    int=5;
    fun(x);
}
```

Inline Function

- A request to the compiler to **replace the function call with the function's actual code** at the point of call
- Avoids the overhead of a normal function call (jumping to another memory location, pushing arguments on the stack, returning)
- Declared using the inline keyword before the function definition
- Best suited for **small, frequently used functions**

Syntax: inline return_type function_name(parameters)

```
{  
    // function body }
```

Example – Inline Function

```
#include <iostream>
using namespace std;
inline int square(int x)
{   return x * x;
}
int main()
{   int result = square(5);
    cout << "Square of " << num << " = " << square(num) << endl;
    return 0;}
```

Overloading Functions

- **Definition:** Defining multiple functions with the **same name but different parameter lists** within the same scope.
- **Distinguishing Factor:** The compiler differentiates between overloaded functions based on the **number, type, and order** of their parameters.
- **Return Type:** The return type alone is *not* sufficient to overload a function.
- **Purpose:** To provide a single, intuitive name for operations that perform similar tasks on different data types or with different numbers of arguments.
- **Example:** `add(int a, int b)`, `add(double a, double b)`, `add(int a, int b, int c)`.

Example - Overloading Functions

```
#include<iostream>
```

```
using namespace std;
```

```
int sum(int a,int b)
```

```
{
```

```
    return a+b;
```

```
}
```

```
float sum(float a,float b)
```

```
{
```

```
    return a+b;
```

```
}
```

```
int sum(int a,int b,int c)
```

```
{
```

```
    return a+b+c;
```

```
}
```

```
int main()
```

```
{
```

```
    cout<<sum(10,5)<<endl;
```

```
    cout<<sum(12.5f,3.4f)<<endl;
```

```
    cout<<sum(10,20,3)<<endl;
```

```
}
```

Task

Write Overloaded Functions for finding Minimum of numbers, implement following functions

- 1. Minimum of 2 integers
- 2. Minimum of 3 integers
- 3. Minimum of 2 floats

Pass by Value v/s Pass by Reference

Pass by Value:

- A **copy of the actual argument's value** is passed to the function's parameter.
- Changes made to the parameter inside the function do *not* affect the original argument.
- Safe, as the original data is protected.
- Can be inefficient for large objects due to copying overhead.

Pass by Reference:

- The **memory address (or reference) of the actual argument is passed** to the function's parameter.
- The parameter becomes an alias for the original argument.
- Changes made to the parameter inside the function *do* affect the original argument.
- Efficient for large objects as no copying occurs.
- **Syntax: `void func(int &x)`**

Example – Call by Value

```
#include<iostream>
using namespace std;
//program for pass by value
void swap(int a,int b)
{
    int temp;
    temp=a;
    a=b;
    b=temp;
    cout<<a<<b<<endl;
}
int main()
{
    int x=10,y=20;
    swap(x,y);
    cout<<x<<y;
    return 0;
}
```

Example – Call by Reference

```
#include<iostream>
using namespace std;
//program for pass by reference
void swap(int &a,int &b)
{
    int temp;
    temp=a;
    a=b;
    b=temp; }
int main()
{
    int x=10,y=20;
    swap(x,y);
    cout<<x<<y;
    return 0; }
```

Default Function Arguments

- **Definition:** Allowing function parameters to have default values. If an argument is not provided during the function call, the default value is used.
- **Syntax:** `returnType functionName(type param1, type param2 = defaultValue, ...);`
- **Rules:**
 - Default arguments must be specified from **right to left** in the parameter list.
 - Once a parameter has a default argument, all subsequent parameters to its right must also have default arguments.
- **Purpose:** To provide flexibility in function calls, allowing a function to be called with fewer arguments. Default arguments are useful for defining overloaded functions.

Guess the output?

```
int add (int x, int y) ---- let it be function 1
```

```
{ return x+y; }
```

```
int add (int x, int y, int z) ---- let it be function 2
```

```
{ return x+y+z; }
```

```
main ()
```

```
{
```

```
int c= add (2,5);
```

```
int c= add (2,5,8);
```

```
int c= add (2,5,0);
```

```
}
```

Why this a Bad Code?

Code Duplication: Two entirely separate functions to do the same job.

Hard to Maintain: To change how “add” works, we have to update it 2 places.

Wasted effort: Writing int z manually passing 0 every time is tedious.

Solution

For example :

```
Int add (int x, int y, int z=0)
{
Return x+y+z;
}
```

Now this function can also be called for two as well as three parameters.

If we pass (2,5) then the value of Z will automatically become 0 as default and when we pass (2,5,8) then Z will become 8 in this case.

Example - Default Function Arguments

```
#include <iostream>
using namespace std;
int sum(int a,int b,int c=0)
{
    return a+b+c;
}
int main()
{
    cout<<sum(10,20,3)<<endl;
    cout<<sum(10,20)<<endl;
    return 0;
}
```

Task

Write a function to find maximum of 3 numbers using default argument

Solution

```
# include<iostream>
using namespace std;
int max(int a=0,int b=0, int c=0)
{
    return a>b && a>c ? a:b>c? b:c;
}
int main()
{
    cout<<max()<<endl;
    cout<<max(10)<<endl;
    cout<<max(10,13)<<endl;
    cout<<max(10,13,15)<<endl;
    return 0;
}
```

```

#include <iostream>
// Function prototype with a default argument
int add(int a, int b, int c = 0);
// Inline function for simple multiplication
inline int multiply(int a, int b) {
    return a * b; }
// Recursive function to calculate factorial
int factorial(int n) {
    if (n <= 1) {
        return 1;
    }
    return n * factorial(n - 1); }
// Overloaded function for adding doubles
double add(double a, double b) {
    return a + b;
}
// Pass by reference function
void increment(int &val) {
    val++;
}
void greet(std::string name, std::string greeting =
"Hello") {
    std::cout << greeting << ", " << name << "!" <<
std::endl; }

```

```

int main() {
    int x = 10;
    std::cout << "add(1, 2) with default arg: " << add(1, 2) <<
std::endl;
    std::cout << "add(1, 2, 3): " << add(1, 2, 3) << std::endl;
    std::cout << "multiply(4, 5): " << multiply(4, 5) << std::endl;
    std::cout << "factorial(5): " << factorial(5) << std::endl;
    std::cout << "overloaded add(1.5, 2.5): " << add(1.5, 2.5) <<
std::endl;
    std::cout << "Before increment: " << x << std::endl;
    increment(x); // Pass by reference
    std::cout << "After increment: " << x << std::endl;
    greet("Alice"); // Uses the default greeting
    greet("Bob", "Hi");

    return 0;
}
// Function definition
int add(int a, int b, int c) {
    return a + b + c;
}

```

Class Declaration & Object Mechanism

Class Declaration:

- Defines the class's interface (public, private, protected members).
- Syntax: `class ClassName { /* members */ };`

Example:

```
class Car {  
    public:  
        void accelerate();  
    private:  
        int speed;  
};
```

Object Mechanism (Instantiation):

- Creating an object (an instance) of a class.
- Syntax: `ClassName objectName;` or `ClassName* objectPtr = new ClassName();`
- When an object is created, memory is allocated for its data members, and its constructor is called.

Objects & References

Objects: As discussed, instances of classes that hold data and provide functionality.

References to Objects:

- An alias (another name) for an existing object.
- Must be initialized at the time of declaration and cannot be re-bound to another object.
- Syntax: `ClassName &referenceName = existingObject;`
- Changes made through the reference directly affect the original object.
- Often used for pass-by-reference to functions to avoid copying large objects.

Constructors and Destructors

Definitions

- **Constructors:**

- Special member functions called automatically when an object is created.
- Have the **same name as the class and no return type**.
- Used to initialize the object's data members and allocate resources.
- Can be overloaded.

- **Destructors:**

- Special member functions called automatically when an object is destroyed (goes out of scope or **delete** is called).
- Have the same name as the class, prefixed with a **tilde (~)**, and **no return type** or parameters.
- Used to release resources allocated by the object (e.g., deallocate memory, close files)

Constructor

A constructor is a special member function with the same name as the class.

Constructor

A **constructor** is a special member function in a class that is automatically called when an object of that class is created. Its primary purpose is to **initialize the object's data members**.

Key Characteristics:

1. **Same Name as the Class:** The constructor's name must be identical to the class name.
2. **No Return Type:** It cannot have a return type, not even **void**.
3. **Automatic Call:** You don't call it directly; it's invoked automatically when an object is instantiated.
4. **Used for Initialization:** Its main job is to put the object into a valid, initial state. This prevents objects from having garbage values.

Characteristics of Constructors

- A constructor should be declared in the **public section** of the class.
- They are automatically invoked whenever the object is created.
- They cannot return values and do not have return types.
- It can have default arguments.

Constructor - Example

```
#include <iostream>
using namespace std;
class A {
public:
    // Constructor without any parameters
    A() {
        cout << "Constructor called" << endl;
    }
};
int main() {
    A obj1;
    return 0;
}
```

Constructor - Example

```
#include <iostream>

class Circle {
private:
    double radius;
public:
    // This is the constructor
    Circle(double r) {
        radius = r;
    }
};

int main() {
    // The constructor is called here with '10.5'
    Circle c1(10.5);
    return 0;
}
```

Find the Output

```
#include <iostream>
using namespace std;
class Car {
public:
    string brand;
    Car() {
        brand = "Unknown";
        cout << "Constructor ran for a
car!" << endl;
    }
};
```

```
int main() {
    cout << "Before creating car" << endl;
    Car c1;
    cout << "After creating car" << endl;

    return 0;
}
```

```
Before creating car
Constructor ran for a car!
After creating car
```

Types of Constructors

- Default Constructor
- Parameterized Constructor
- Copy Constructor
- Explicit Constructor

Types of Constructors

1. Default Constructor

A default constructor either takes no arguments or has default values for all its parameters. If you do not write any constructor, the **compiler automatically generates an implicit, empty default constructor** for you.

```
class Player
{ public: int score; // Default constructor
Player()
{ score = 0;
} };
```

Default Constructor - Example

```
#include <iostream>
using namespace std;
// Class with no explicitly defined constructors
class A {
public:
};
int main() {
    // Creating object
    A a;
    return 0;
}
```

Types of Constructors

2. Parameterized Constructor

This constructor accepts arguments, allowing you to pass custom values to your object's variables at the exact moment of creation.

```
class Player
{ public: int score; // Parameterized constructor
  Player(int starting_score)
  {
    score = starting_score;
  } };
```

Parameterized Constructor - Example

```
#include <iostream>

using namespace std;

class A {
public:
    int val;

    // Parameterized Constructor
    A(int x) {
        val = x;
    }
};

int main() {
    // Creating object with a parameter
    A a(10);
    cout << a.val;
    return 0;
}
```

Find the Output

```
class Car {  
public:  
    string brand;  
    Car(string b) {        // takes an argument now  
        brand = b;  
        cout << "Made a " << brand << endl;  
    }  
};  
  
int main() {  
    Car c1("Honda");  
    Car c2("Tesla");  
}
```

```
Made a Honda  
Made a Tesla
```

Types of Constructors

3. Copy Constructor

- **Definition:** A special constructor that initializes a **new object** as a copy of an **existing object** of the same class.
- **Signature:** `ClassName(const ClassName& other);`
- **When called:**
 - When an object is initialized by another object of the same class (`MyClass obj2 = obj1;` or `MyClass obj2(obj1);`).
 - When an object is passed by value to a function.
 - When an object is returned by value from a function.

Types of Constructors

3. Copy Constructor

A copy constructor initializes a brand-new object by copying the member variables of an already existing object of the same class.

```
class Player
{
public: int score; // Copy constructor (takes a reference to another object)
Player(const Player &other)
{ score = other.score;
} };
```

Relevance to Copy Constructor

The copy constructor often takes a `const` reference to an object as its parameter.

- `ClassName(const ClassName& other);`

Why `const` reference?

- **Efficiency:** Avoids unnecessary copying of the object being copied (which would recursively call the copy constructor).
- **Safety:** Ensures that the **original object is not modified** during the copy operation.

This reinforces the use of references for efficient and safe object handling in C++.

```
#include <iostream>

using namespace std;

class Complex {
private:
    int real;

    int imag;
public:
    Complex (int r, int i) {
        real = r;

        imag = i; }
    Complex (const Complex &c) {
        real = c.real;

        imag = c.imag; }}
```

```
void display()
{
    cout << real << "+" << imag << "i"
<< endl;
}

};

int main()
{
    Complex c1(3, 4);
    Complex c2 = c1; // Copy constructor is
called here
    c1.display();
    c2.display();
}
```

Practice Problem

BankAccount class —

default constructor sets balance to 0;

parameterized constructor takes account number and initial balance;

copy constructor duplicates an account (e.g., for a joint account demo).

Explicit Constructor

- **Definition:** A constructor declared with the **explicit** keyword.
- **Purpose:** To prevent implicit type conversions from the constructor's argument type to the class type.
- **Why use **explicit**?**
 - To avoid unintended conversions that might lead to logical errors or performance overhead.
 - To make code clearer by requiring explicit casting when a conversion is desired.

Example

```
#include <iostream>
using namespace std;
class Complex {
public:
    int real;
    int imag;
    Complex (int r=0, int i=0)
    {
        real = r;
        imag = i;
    }
    Complex (const Complex &c)
    {
        real = c.real;
        imag = c.imag;
    }
};
```

```
void display(Complex c)
{
    cout << c.real << "+" << c.imag << "i"
    << endl;
}

int main()
{
    Complex c1;
    display(c1);
}
```

Example - Solution

```
#include <iostream>
using namespace std;
class Complex {
public:
    int real;
    int imag;
    explicit Complex (int r=0, int i=0)
    {
        real = r;
        imag = i;
    }
    Complex (const Complex &c)
    {
        real = c.real;
        imag = c.imag;
    }
};
```

```
void display(Complex c) //c=10 passing object
look for constructor with int parameter
{
    cout << c.real << "+" << c.imag << "i"
    << endl;
}

int main()
{
    Complex c1;
    display(Complex(10)); //c=10 passing
    object look for constructor with int parameter
}
```

Compiler Synthesized Constructor

- **Definition:** If you don't explicitly define any constructors for a class, the C++ compiler automatically generates (synthesizes) a **default constructor**.
- **Characteristics:**
 - It's a **public, no-argument constructor**.
 - It performs **default initialization** for built-in types (leaves them uninitialized) and calls the default constructors for member objects.
 - It does **not** initialize dynamically allocated memory or perform any specific setup.
- **When it's generated:** Only if no user-defined constructors (default, copy, move, or parameterized) are provided.

Note

if you write *any* constructor yourself, C++ stops generating the default no-argument constructor automatically.

So if you have `Student(int r)` and

try `Student s1;`

with no arguments, it will give a compile error — unless you also explicitly write a no-argument version.

Overloaded Constructors

Definition: A class can have **multiple constructors**, each with a **different parameter list**.

Purpose: To provide **different ways to initialize an object**, depending on the **available input** or desired initial state.

Compiler Resolution: The compiler determines which constructor to call based on the number and types of arguments provided during object creation.

Example:

```
class Point {  
    public:  
        Point(); // Default constructor  
        Point(int x, int y); // Parameterized constructor  
};
```

std::cin

The `std::cin` object in C++ is a standard input stream.

It's an object of the `istream` class and is used to read data from the standard input device, which is typically your keyboard.

The `cin` part stands for "**character in.**"

How It Works

`std::cin` works with the **extraction operator** `>>`. This operator is used to extract data from the input stream and store it in a variable. The operator automatically handles the conversion of the input data type to the type of the variable it's assigned to.

```
#include <iostream>
#include <string>
```

```
int main() {
    // Declare a variable to store the user's name
    std::string name;

    // Prompt the user to enter their name
    std::cout << "Please enter your name: ";

    // Use std::cin to read the name from the keyboard
    std::cin >> name;

    // Declare a variable to store the user's age
    int age;

    // Prompt the user to enter their age
    std::cout << "Please enter your age: ";

    // Use std::cin to read the age from the keyboard
    std::cin >> age;

    // Display the stored name and age
    std::cout << "Hello, " << name << "! You are " << age << " years old." << std::endl;

    return 0;
}
```

Task

Create a Ticket class with seatNumber and price.

Requirements:

Default constructor: seatNumber = 0, price = 0.0

Parameterized constructor: takes seat number and price

Explicit constructor: takes only a seat number, assumes a fixed standard price (e.g., 200)

Copy constructor: prints "Duplicate ticket issued for seat [seatNumber]"

Write main() to book 3 tickets using different constructors, then duplicate one ticket and show both tickets are independent.